

Skill

Task 1 : To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components.

```
vennela_07@VennelaPC:~$ mkdir integrated_shell
vennela_07@VennelaPC:~$ cd integrated_shell
vennela_07@VennelaPC:~/integrated_shell$ nano integrated_shell.c
vennela_07@VennelaPC:~/integrated_shell$ gcc integrated_shell.c -o integrated_shell
vennela_07@VennelaPC:~/integrated_shell$ ./integrated_shell
=====
      INTEGRATED MINI SHELL
=====
Supported commands:
pwd
ls
date
echo <message>
exit

shell> pwd
/home/vennela_07/integrated_shell
Process completed with exit code: 0

shell> ls
integrated_shell  integrated_shell.c
Process completed with exit code: 0

shell> date
Mon Sep 28 03:56:26 UTC 2026
Process completed with exit code: 0

shell> echo Hello Linux
Hello Linux
Process completed with exit code: 0

shell> abc
Command execution failed: No such file or directory
Process completed with exit code: 1

shell> 
Empty command. Try again.
Command execution failed: No such file or directory
Process completed with exit code: 1

shell> exit
Exiting shell...
Shell terminated successfully.
```

```
GNU nano 7.2                                     integrated_shell.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>

/*
Module 1: Command Parser
*/
int parse_command(char *input, char *args[])
{
    int count = 0;

    char *token = strtok(input, " \\t\\n");

    while (token != NULL && count < 9)
    {
        args[count++] = token;
        token = strtok(NULL, " \\t\\n");
    }

    args[count] = NULL;

    return count;
}

/*
Module 2: Command Validator
*/
int validate_command(char *args[])
{
    if (args[0] == NULL)
    {
        return 0;
    }

    if (strcmp(args[0], "exit") == 0)
    {
        return 1;
    }

    if (strcmp(args[0], "pwd") == 0)
    {
        return 1;
    }

    if (strcmp(args[0], "ls") == 0)
    {
        return 1;
    }

    if (strcmp(args[0], "date") == 0)
    {
        return 1;
    }

    if (strcmp(args[0], "echo") == 0)
    {
        return 1;
    }
}
```

```

----- */
int main()
{
    char input[256];
    char *args[10];

    printf("=====\n");
    printf("      INTEGRATED MINI SHELL\n");
    printf("=====\n");

    printf("Supported commands:\n");
    printf("pwd\n");
    printf("ls\n");
    printf("date\n");
    printf("echo <message>\n");
    printf("exit\n");

    while (1)
    {
        printf("\nshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            printf("\nInput error or EOF.\n");
            break;
        }

        /* Parse command */
        int count = parse_command(input, args);

        if (count == 0)
        {
            printf("Empty command. Try again.\n");
            continue;
        }

        /* Validate interface */
        if (!validate_command(args))
        {
            printf("Invalid command.\n");
            continue;
        }

        /* Exit command */
        if (strcmp(args[0], "exit") == 0)
        {
            printf("Exiting shell...\n");
            break;
        }

        /* Execute command */
        execute_command(args);
    }

    printf("Shell terminated successfully.\n");

    return 0;
}

```

Task 2 : To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios.

```
vennela_07@VennelaPC:~$ mkdir runtime_errors
vennela_07@VennelaPC:~$ cd runtime_errors
vennela_07@VennelaPC:~/runtime_errors$ nano runtime_errors.c
vennela_07@VennelaPC:~/runtime_errors$ gcc runtime_errors.c -o runtime_errors
vennela_07@VennelaPC:~/runtime_errors$ ./runtime_errors
=====
    RUNTIME ERROR HANDLING SHELL
=====
Commands: pwd, ls, date, echo, exit

shell> pwd
/home/vennela_07/runtime_errors
Command completed successfully.

shell> hello
Command execution failed: No such file or directory
ERROR: Command 'hello' exited with code 1

shell> ERROR: Invalid syntax: empty command
Command execution failed: No such file or directory
ERROR: Command 'ERROR:' exited with code 1

shell> date
Mon Sep 28 04:00:05 UTC 2026
Command completed successfully.

shell> exit
Exiting safely...
Shell closed safely.
vennela_07@VennelaPC:~/runtime_errors$ ./runtime_errors
=====
    RUNTIME ERROR HANDLING SHELL
=====
Commands: pwd, ls, date, echo, exit

shell> abc
Command execution failed: No such file or directory
```

```
shell> xyz
Command execution failed: No such file or directory
ERROR: Command 'xyz' exited with code 1

shell> exit
Exiting safely...
Shell closed safely.
vennela_07@VennelaPC:~/runtime_errors$ ./runtime_errors
=====
    RUNTIME ERROR HANDLING SHELL
=====
Commands: pwd, ls, date, echo, exit
```

```

GNU nano 7.2                                     runtime_errors.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <errno.h>
#include <time.h>

/* Log errors into a file */
void log_error(const char *message)
{
    FILE *file;
    time_t now;

    file = fopen("error.log", "a");

    if (file == NULL)
    {
        perror("Cannot open error.log");
        return;
    }

    now = time(NULL);

    fprintf(file, "[%s] %s\n", ctime(&now), message);

    fclose(file);
}

/* Display and log an error */
void report_error(const char *message)
{
    printf("ERROR: %s\n", message);
    log_error(message);
}

/* Execute a command */
void execute_command(char *args[])
{
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0)
    {

```

```

        if (waitpid(pid, &status, 0) < 0)
        {
            report_error("waitpid() failed");
            return;
        }

        if (WIFEXITED(status))
        {
            int code = WEXITSTATUS(status);

            if (code != 0)
            {
                char message[200];

                snprintf(message, sizeof(message),
                         "Command '%s' exited with code %d",
                         args[0], code);

                report_error(message);
            }
        }
    }
}

```

```

fflush(stdout);

if (fgets(input, sizeof(input), stdin) == NULL)
{
    report_error("Input reading failed");
    break;
}

/* Detect empty input */
if (strlen(input) <= 1)
{
    report_error("Invalid syntax: empty command");
    continue;
}

int count = parse_command(input, args);

if (count == 0)
{
    report_error("Invalid syntax");
    continue;
}

/* Exit */
if (strcmp(args[0], "exit") == 0)
{
    printf("Exiting safely...\n");
    break;
}

/* Check command length */
if (count >= 9)
{
    report_error("Too many command arguments");
    continue;
}

/* Execute command */
execute_command(args);
}

printf("Shell closed safely.\n");

return 0;
}

```

Task 3 : To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings.

```
vennela_07@VennelaPC:~$ memory_check.c
memory_check.c: command not found
vennela_07@VennelaPC:~$ mkdir valgrind_memory
vennela_07@VennelaPC:~$ cd valgrind_memory
vennela_07@VennelaPC:~/valgrind_memory$ nano memory_check.c
```

```
vennela_07@VennelaPC:~/valgrind_memory$ gcc -g memory_check.c -o memory_check
vennela_07@VennelaPC:~/valgrind_memory$ ./memory_check
Memory allocation started.
Memory allocated successfully.
First array: 1 2 3 4 5
Second array: 10 11 12 13 14 15 16 17 18 19
Program finished.
vennela_07@VennelaPC:~/valgrind_memory$ valgrind --version
valgrind-3.22.0
vennela_07@VennelaPC:~/valgrind_memory$ valgrind --leak-check=full ./memory_check
==5905== Memcheck, a memory error detector
==5905== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==5905== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==5905== Command: ./memory_check
==5905==
Memory allocation started.
Memory allocated successfully.
First array: 1 2 3 4 5
Second array: 10 11 12 13 14 15 16 17 18 19
Program finished.
==5905==
==5905== HEAP SUMMARY:
==5905==   in use at exit: 40 bytes in 1 blocks
==5905== total heap usage: 3 allocs, 2 frees, 1,084 bytes allocated
==5905==
==5905== 40 bytes in 1 blocks are definitely lost in loss record 1 of 1
==5905==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==5905==    by 0x1091FB: main (memory_check.c:12)
==5905==
==5905== LEAK SUMMARY:
==5905==   definitely lost: 40 bytes in 1 blocks
==5905==   indirectly lost: 0 bytes in 0 blocks
==5905==   possibly lost: 0 bytes in 0 blocks
==5905==   still reachable: 0 bytes in 0 blocks
==5905==   suppressed: 0 bytes in 0 blocks
==5905==
==5905== For lists of detected and suppressed errors, rerun with: -s
==5905== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
vennela_07@VennelaPC:~/valgrind_memory$ valgrind --leak-check=full ./memory_check
==5940== Memcheck, a memory error detector
```

```

==5940== definitely lost: 40 bytes in 1 blocks
==5940== indirectly lost: 0 bytes in 0 blocks
==5940== possibly lost: 0 bytes in 0 blocks
==5940== still reachable: 0 bytes in 0 blocks
==5940== suppressed: 0 bytes in 0 blocks
==5940==
==5940== For lists of detected and suppressed errors, rerun with: -s
==5940== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
vennela_07@VennelaPC:~/valgrind_memory$ nano memory_check.c
vennela_07@VennelaPC:~/valgrind_memory$ gcc -g memory_check.c -o memory_check
vennela_07@VennelaPC:~/valgrind_memory$ ./memory_check
Memory allocation started.
Memory allocated successfully.
First array: 1 2 3 4 5
Second array: 10 11 12 13 14 15 16 17 18 19
Program finished.
vennela_07@VennelaPC:~/valgrind_memory$ valgrind --leak-check=full --show-leak-kinds=all ./memory_check
==6632== Memcheck, a memory error detector
==6632== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==6632== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==6632== Command: ./memory_check
==6632==
Memory allocation started.
Memory allocated successfully.
First array: 1 2 3 4 5
Second array: 10 11 12 13 14 15 16 17 18 19
Program finished.
==6632==
==6632== HEAP SUMMARY:
==6632== in use at exit: 0 bytes in 0 blocks
==6632== total heap usage: 3 allocs, 3 frees, 1,084 bytes allocated
==6632==
==6632== All heap blocks were freed -- no leaks are possible
==6632==
==6632== For lists of detected and suppressed errors, rerun with: -s
==6632== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
vennela_07@VennelaPC:~/valgrind_memory$ valgrind --leak-check=full ./memory_check > valgrind_report.txt 2>&1
vennela_07@VennelaPC:~/valgrind_memory$ ls
memory_check  memory_check.c  valgrind_report.txt

```

```

GNU nano 7.2 memory_check.c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *a;
    int *b;

    printf("Memory allocation started.\n");

    a = malloc(5 * sizeof(int));
    b = malloc(10 * sizeof(int));

    if (a == NULL || b == NULL)
    {
        printf("Memory allocation failed.\n");
        free(a);
        free(b);
        return 1;
    }

    for (int i = 0; i < 5; i++)
    {
        a[i] = i + 1;
    }

    for (int i = 0; i < 10; i++)
    {
        b[i] = i + 10;
    }

    printf("Memory allocated successfully.\n");

    printf("First array: ");
    for (int i = 0; i < 5; i++)

```

```

    for (int i = 0; i < 5; i++)
    {
        a[i] = i + 1;
    }

    for (int i = 0; i < 10; i++)
    {
        b[i] = i + 10;
    }

    printf("Memory allocated successfully.\n");

    printf("First array: ");
    for (int i = 0; i < 5; i++)
    {
        printf("%d ", a[i]);
    }

    printf("\nSecond array: ");
    for (int i = 0; i < 10; i++)
    {
        printf("%d ", b[i]);
    }

    printf("\n");

    /* Free only a */
    free(a);
    free(b);
    printf("Program finished.\n");

    return 0;
}

```

Task 4 : To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports.

```

vennela_07@VennelaPC:~$ mkdir automated_testing
vennela_07@VennelaPC:~$ cd automated_testing
vennela_07@VennelaPC:~/automated_testing$ nano calculator.c

vennela_07@VennelaPC:~/automated_testing$ gcc calculator.c -o calculator
vennela_07@VennelaPC:~/automated_testing$ ./calculator
Enter expression: 10 + 5
Result: 15
vennela_07@VennelaPC:~/automated_testing$ nano test_script.sh
vennela_07@VennelaPC:~/automated_testing$ chmod +x test_script.sh
vennela_07@VennelaPC:~/automated_testing$ ./test_script.sh
Running automated tests...
TC01 : PASS
TC02 : PASS
TC03 : PASS
TC04 : PASS
TC05 : PASS
TC06 : PASS

```

```

Total Passed: 6
Total Failed: 0

Test report generated: test_report.txt
vennela_07@VennelaPC:~/automated_testing$ cat test_report.txt
=====
                AUTOMATED TEST REPORT
=====

TC01 : PASS
TC02 : PASS
TC03 : PASS
TC04 : PASS
TC05 : PASS
TC06 : PASS

=====
Total Passed: 6
Total Failed: 0
=====
vennela_07@VennelaPC:~/automated_testing$ ls
calculator  calculator.c  test_report.txt  test_script.sh

```

```

GNU nano 7.2 calculator.c
#include <stdio.h>

int main()
{
    int a, b;
    char op;

    printf("Enter expression: ");
    scanf("%d %c %d", &a, &op, &b);

    if (op == '+')
        printf("Result: %d\n", a + b);

    else if (op == '-')
        printf("Result: %d\n", a - b);

    else if (op == '*')
        printf("Result: %d\n", a * b);

    else if (op == '/')
    {
        if (b == 0)
        {
            printf("Error: Division by zero\n");
            return 1;
        }

        printf("Result: %d\n", a / b);
    }
    else
    {
        printf("Error: Invalid operator\n");
        return 1;
    }

    return 0;
}

```

```

GNU nano 7.2 test_script.sh
#!/bin/bash

PROGRAM="./calculator"
REPORT="test_report.txt"

echo "===== " > $REPORT
echo "      AUTOMATED TEST REPORT" >> $REPORT
echo "===== " >> $REPORT
echo "" >> $REPORT

pass=0
fail=0

run_test()
{
    test_id=$1
    input=$2
    expected=$3

    actual=$(echo "$input" | $PROGRAM 2>&1)

    if echo "$actual" | grep -q "$expected"
    then
        echo "$test_id : PASS"
        echo "$test_id : PASS" >> $REPORT
        pass=$((pass + 1))
    else
        echo "$test_id : FAIL"
        echo "$test_id : FAIL" >> $REPORT

        echo "Expected: $expected" >> $REPORT
        echo "Actual: $actual" >> $REPORT

        fail=$((fail + 1))
    fi
}

echo "Running automated tests..."

run_test "TC01" "10 + 5" "Result: 15"
run_test "TC02" "10 - 5" "Result: 5"
run_test "TC03" "10 * 5" "Result: 50"
run_test "TC04" "10 / 5" "Result: 2"
run_test "TC05" "10 / 0" "Error: Division by zero"
run_test "TC06" "10 % 5" "Error: Invalid operator"

echo "" >> $REPORT
echo "===== " >> $REPORT
echo "Total Passed: $pass" >> $REPORT
echo "Total Failed: $fail" >> $REPORT
echo "===== " >> $REPORT

echo ""
echo "Total Passed: $pass"
echo "Total Failed: $fail"

echo ""
echo "Test report generated: $REPORT"

```


